

Enterprise Carpooling Platform

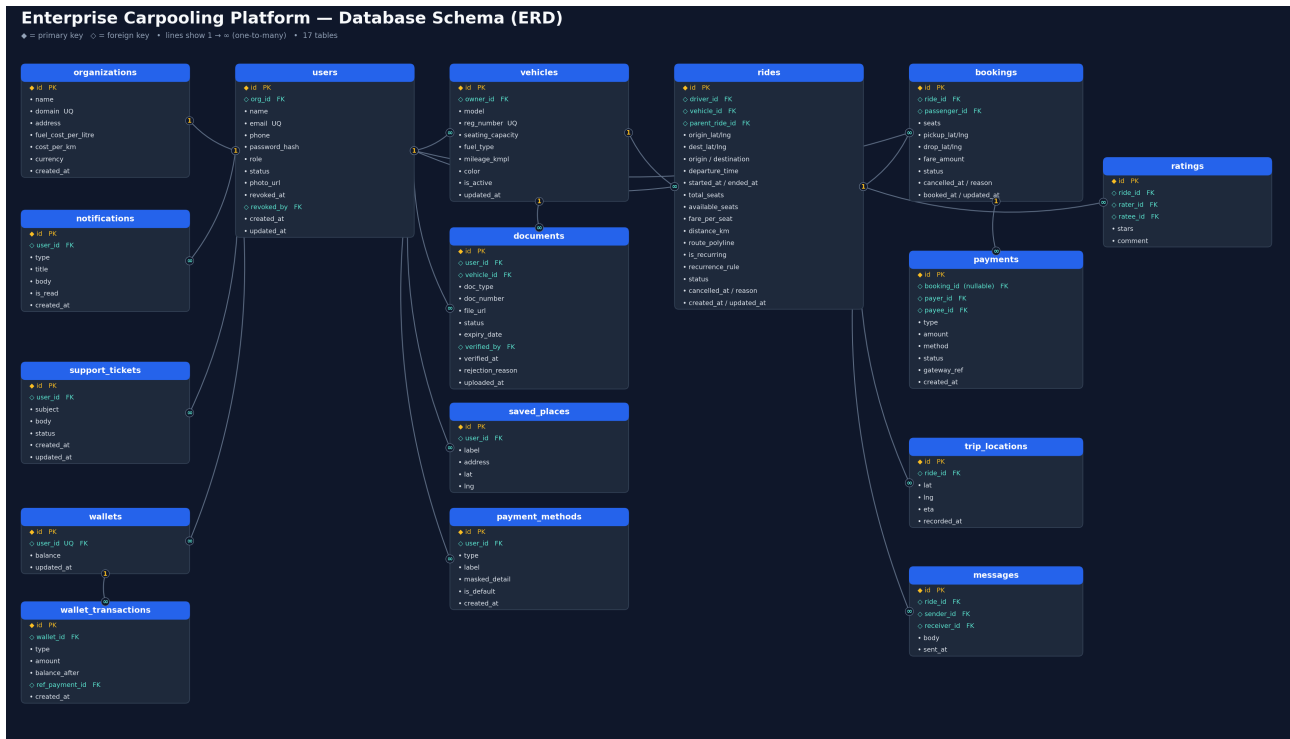
Database Schema & Technical Design

This document defines the relational data model for an enterprise ride-sharing platform where employees of registered organizations discover, offer, book, track, and pay for shared rides. It covers every table, its columns, keys, constraints, and the enumerations that drive the trip and payment lifecycle, followed by a recommended technology stack for the hackathon build.

Scope	Multi-tenant (multiple organizations)
Engine	PostgreSQL 15+ (+ PostGIS for matching)
Stack	FastAPI (Python) · React · PostgreSQL+PostGIS · free OSM maps — CONFIRMED
Modules	Auth · KYC/Docs · Rides · Bookings · Trips · Live Tracking · Wallet/Payments · Reports
Tables	17 core entities

1 · Entity Relationship Diagram

The diagram below shows all entities and their relationships. Diamonds (◆) mark primary keys and open diamonds (◻) mark foreign keys. Every connector represents a one-to-many relationship ($1 \rightarrow \infty$).



2 · Schema Review — Gaps Closed & Fixes Applied

A close re-read of the problem statement surfaced **functional gaps** (features the brief requires that the first draft could not represent) plus several **integrity** gaps. This revision closes both. The entity set grows from 14 to 17 tables (adds **documents**, **payment_methods**, **support_tickets**).

Gap / Issue	Fix applied	Type
Drivers must upload & verify a driving licence before publishing rides (KYC).	NEW documents table (licence, ID, vehicle RC/insurance) with pending/verified/rejected + expiry + admin verifier.	Feature
Admin 'provides access' / revokes an employee — no way to represent it.	users.status enum(invited/active/suspended) + revoked_at + revoked_by; checked on login & each request.	Feature
Reports need Fuel Consumption / Cost-per-km / Efficiency — no mileage stored.	Added vehicles.mileage_kmpl; fuel used = distance ÷ mileage × org fuel price.	Feature
Wallet Recharge is a payment with no booking, but booking_id was NOT NULL.	booking_id now nullable; added payments.type; partial-unique only for ride payments.	Feature
Trip lifecycle & reports need ACTUAL start/end times (only scheduled existed).	Added rides.started_at / ended_at (nullable).	Feature
Settings screens 'Payment Methods' and 'Help & Support' had no storage.	NEW payment_methods (tokenised display) and support_tickets tables.	Feature
Ride Cancellation (bonus) had no metadata.	Added cancelled_at + cancel_reason to rides and bookings; restore seats on cancel.	Feature
A passenger could book the same ride twice.	UNIQUE(ride_id, passenger_id) on bookings.	Integrity
available_seats could drift below 0 / above total_seats.	CHECK ($0 \leq \text{available_seats} \leq \text{total_seats}$); decrement in a transaction.	Integrity
Proximity ride-matching would table-scan raw lat/long.	PostGIS geography + GiST index (or bounding-box + composite index).	Integrity
Money/rating integrity + audit trail.	All money numeric(,2), CHECK ≥ 0 , stars 1–5; updated_at on mutable tables.	Integrity

3 · Table Definitions

Types use PostgreSQL notation. **PK** = primary key, **FK** = foreign key, **UQ** = unique, **NN** = not null.

organizations

A registered company / tenant. Every user and ride belongs to one org.

Column	Type	Key / Constraint	Notes
id	uuid	PK	Tenant identifier
name	varchar(150)	NN	Company name
domain	varchar(120)	UQ	Email domain used to verify employees
address	text		Head-office address
fuel_cost_per_litre	numeric(8,2)		Admin-set, feeds cost reports
cost_per_km	numeric(8,2)		Admin-set travel cost baseline
currency	char(3)		ISO code, e.g. INR
created_at	timestampz	NN	default now()

users

An employee. The same user can be a driver and a passenger.

Column	Type	Key / Constraint	Notes
id	uuid	PK	
org_id	uuid	FK→organizations	Tenant scope
name	varchar(120)	NN	
email	varchar(160)	UQ, NN	Login identifier
phone	varchar(20)		For call feature
password_hash	text	NN	bcrypt/argon2
role	enum(user_role)	NN	'admin' 'employee'
status	enum(user_status)	NN	invited active suspended (access control)
photo_url	text		Profile picture
revoked_at	timestampz		Set when admin suspends access
revoked_by	uuid	FK→users	The admin who revoked; nullable
created_at	timestampz	NN	default now()
updated_at	timestampz	NN	auto-touch on update

vehicles

Vehicles owned by an employee. Required before offering a ride.

Column	Type	Key / Constraint	Notes
id	uuid	PK	
owner_id	uuid	FK→users	
model	varchar(120)	NN	e.g. Honda City
reg_number	varchar(20)	UQ, NN	License plate
seating_capacity	smallint	NN	Total seats, CHECK > 0

Column	Type	Key / Constraint	Notes
fuel_type	enum(fuel_type)		'petrol' 'diesel' 'ev' 'cng'
mileage_kmpl	numeric(5,2)		Fuel efficiency (km/litre); CHECK > 0 — feeds fuel & cost reports
color	varchar(30)		
is_active	boolean	NN	Soft-retire without deleting
updated_at	timestampz	NN	auto-touch on update

saved_places

Frequently used pickup/drop points (Home, Office, ...).

Column	Type	Key / Constraint	Notes
id	uuid	PK	
user_id	uuid	FK→users	
label	varchar(60)	NN	'Home', 'Office', custom
address	text	NN	
lat	numeric(9,6)	NN	
lng	numeric(9,6)	NN	

rides

A ride offered by a driver. Passengers book seats against it.

Column	Type	Key / Constraint	Notes
id	uuid	PK	
driver_id	uuid	FK→users	
vehicle_id	uuid	FK→vehicles	
parent_ride_id	uuid	FK→rides (self)	Links recurring instances to a template; NULL for one-off
origin	text	NN	Pickup label
origin_lat / origin_lng	numeric(9,6)	NN	Start coordinates
destination	text	NN	Drop label
dest_lat / dest_lng	numeric(9,6)	NN	End coordinates
departure_time	timestampz	NN	Scheduled start
started_at	timestampz		Actual trip start (driver taps Start)
ended_at	timestampz		Actual completion; gives real duration
total_seats	smallint	NN	Seats offered
available_seats	smallint	NN	CHECK $0 \leq \text{available_seats} \leq \text{total_seats}$
fare_per_seat	numeric(8,2)	NN	CHECK ≥ 0
distance_km	numeric(8,2)		From maps API, feeds reports
route_polyline	text		Encoded route for map redraw
is_recurring	boolean	NN	default false
recurrence_rule	varchar(120)		iCal RRULE / weekday mask
status	enum(ride_status)	NN	See lifecycle below
cancelled_at	timestampz		Set on cancellation

Column	Type	Key / Constraint	Notes
cancel_reason	varchar(200)		Free-text reason
created_at	timestampz	NN	default now()
updated_at	timestampz	NN	auto-touch on update

bookings

A passenger's reservation of one or more seats on a ride.

Column	Type	Key / Constraint	Notes
id	uuid	PK	
ride_id	uuid	FK→rides	UNIQUE(ride_id, passenger_id) — no double-booking
passenger_id	uuid	FK→users	
seats	smallint	NN	CHECK > 0
pickup_lat / pickup_lng	numeric(9,6)		Passenger pickup
drop_lat / drop_lng	numeric(9,6)		Passenger drop
fare_amount	numeric(10,2)	NN	seats × fare_per_seat
status	enum(booking_status)	NN	'booked' 'cancelled' 'completed'
cancelled_at	timestampz		Set on cancellation; restore seats
cancel_reason	varchar(200)		Free-text reason
booked_at	timestampz	NN	default now()
updated_at	timestampz	NN	auto-touch on update

trip_locations

Time-series GPS pings for live tracking while a trip is active.

Column	Type	Key / Constraint	Notes
id	bigserial	PK	
ride_id	uuid	FK→rides	
lat	numeric(9,6)	NN	
lng	numeric(9,6)	NN	
eta	integer		Seconds to destination
recorded_at	timestampz	NN	Ping timestamp

payments

A payment: either a ride settlement OR a wallet top-up (Razorpay test / cash / wallet).

Column	Type	Key / Constraint	Notes
id	uuid	PK	
booking_id	uuid	FK→bookings	NULL for wallet recharge. Partial UNIQUE WHERE type='ride_payment'
payer_id	uuid	FK→users	Passenger (or wallet owner)
payee_id	uuid	FK→users	Driver; NULL for recharge
type	enum(payment_type)	NN	'ride_payment' 'wallet_recharge'
amount	numeric(10,2)	NN	CHECK ≥ 0

Column	Type	Key / Constraint	Notes
method	enum(pay_method)	NN	'cash' 'card' 'upi' 'wallet'
status	enum(pay_status)	NN	'pending' 'success' 'failed'
gateway_ref	varchar(120)		Razorpay payment id; nullable for cash
created_at	timestampz	NN	default now()

documents

Driver/vehicle KYC documents. A verified driving licence is required before publishing rides.

Column	Type	Key / Constraint	Notes
id	uuid	PK	
user_id	uuid	FK→users	Document owner (driver)
vehicle_id	uuid	FK→vehicles	Set for vehicle docs (RC/insurance); NULL for personal
doc_type	enum(doc_type)	NN	driving_license id_proof vehicle_rc vehicle_insurance
doc_number	varchar(60)		Licence / document number
file_url	text	NN	Uploaded scan/photo
status	enum(doc_status)	NN	pending verified rejected (default pending)
expiry_date	date		Licence/insurance expiry; blocks if past
verified_by	uuid	FK→users	Admin who verified; nullable
verified_at	timestampz		
rejection_reason	varchar(200)		Why rejected
uploaded_at	timestampz	NN	default now()

payment_methods

Saved payment instruments shown on the Settings → Payment Methods screen.

Column	Type	Key / Constraint	Notes
id	uuid	PK	
user_id	uuid	FK→users	
type	enum(pm_type)	NN	'card' 'upi'
label	varchar(60)		e.g. 'HDFC Credit'
masked_detail	varchar(40)		Tokenised display only (e.g. **** 4242) — Razorpay holds real data
is_default	boolean	NN	default false
created_at	timestampz	NN	default now()

support_tickets

Help & Support requests raised from Settings.

Column	Type	Key / Constraint	Notes
id	uuid	PK	
user_id	uuid	FK→users	

Column	Type	Key / Constraint	Notes
subject	varchar(150)	NN	
body	text	NN	
status	enum(ticket_status)	NN	open in_progress closed
created_at	timestampz	NN	default now()
updated_at	timestampz	NN	auto-touch on update

wallets

One prepaid wallet per user.

Column	Type	Key / Constraint	Notes
id	uuid	PK	
user_id	uuid	FK→users, UQ	
balance	numeric(12,2)	NN	default 0
updated_at	timestampz	NN	

wallet_transactions

Ledger of every wallet credit/debit.

Column	Type	Key / Constraint	Notes
id	bigserial	PK	
wallet_id	uuid	FK→wallets	
type	enum(wtxn_type)	NN	'recharge' 'debit' 'credit'
amount	numeric(12,2)	NN	
balance_after	numeric(12,2)	NN	Running balance
ref_payment_id	uuid	FK→payments	Nullable link
created_at	timestampz	NN	default now()

messages

In-trip chat between driver and passengers.

Column	Type	Key / Constraint	Notes
id	bigserial	PK	
ride_id	uuid	FK→rides	
sender_id	uuid	FK→users	
receiver_id	uuid	FK→users	
body	text	NN	
sent_at	timestampz	NN	default now()

notifications

Push / in-app notifications (bonus feature).

Column	Type	Key / Constraint	Notes
id	bigserial	PK	
user_id	uuid	FK→users	

Column	Type	Key / Constraint	Notes
type	varchar(50)	NN	'booking' 'trip' 'payment'...
title	varchar(150)	NN	
body	text		
is_read	boolean	NN	default false
created_at	timestampz	NN	default now()

ratings

Optional post-trip rating between participants.

Column	Type	Key / Constraint	Notes
id	uuid	PK	
ride_id	uuid	FK→rides	
rater_id	uuid	FK→users	
ratee_id	uuid	FK→users	
stars	smallint	NN	1–5
comment	text		

4 · Enumerations & Lifecycle

The trip lifecycle from the problem statement maps to two coordinated status fields: **rides.status** (the journey) and **payments.status** / **bookings.status** (settlement).

Enum	Allowed values
ride_status	scheduled → started → in_progress → completed (or cancelled)
booking_status	booked · cancelled · completed
pay_status	pending · success · failed
pay_method	cash · card · upi · wallet
payment_type	ride_payment · wallet_recharge
user_role	admin · employee
user_status	invited · active · suspended
doc_type	driving_license · id_proof · vehicle_rc · vehicle_insurance
doc_status	pending · verified · rejected
pm_type	card · upi
ticket_status	open · in_progress · closed
fuel_type	petrol · diesel · ev · cng
wtxn_type	recharge · debit · credit

Key business rules enforced by the schema

- Multi-tenant: users.org_id scopes every query; ride matching only joins users of the same org.
- Access control: only users with status='active' can log in / call the API; login and every request re-check status.
- Publishing a ride requires (a) status='active', (b) a verified, non-expired driving_license in documents, and (c) at least one is_active vehicle.
- available_seats is decremented on booking (in a transaction) and restored on cancellation.
- trip_locations rows are only written while rides.status = 'started'/'in_progress'.
- One ride payment per booking (partial unique); wallet recharge is a payment with type='wallet_recharge' + a wallet_transactions credit.
- Reports: fuel used = distance_km ÷ vehicles.mileage_kmpl; cost = fuel_used × organizations.fuel_cost_per_litre; actual distance/duration come from rides.started_at/ended_at.

Recommended indexes

Index	Purpose
users(email)	UNIQUE — login lookup
users(org_id, status)	Tenant scope + admin participation/access monitoring
documents(user_id, doc_type, status)	Licence-verified check before publishing
payment_methods(user_id)	Settings → Payment Methods list
rides(status, departure_time)	Find-a-ride search filter
rides(origin_lat, origin_lng) / GiST(geog)	Proximity matching (PostGIS)

bookings(passenger_id)	My Trips (passenger view)
bookings(ride_id)	Driver's passenger list
trip_locations(ride_id, recorded_at)	Live-tracking replay / latest ping
payments(booking_id)	UNIQUE — one payment per booking
wallet_transactions(wallet_id, created_at)	Wallet statement

5 · Recommended Technology Stack

Confirmed direction: **Python backend, React frontend**. This stack is optimised for a hackathon — fast to build, strong real-time support, runs fully on localhost, and every third-party service has a free or sandbox tier.

Layer	Recommended	Why
Frontend	React + Vite + TypeScript, Tailwind CSS	Fast dev server; quick to build forms, dashboards and the map screens.
Map render	React-Leaflet + OpenStreetMap tiles	Free, no API key locally; draws route, pickup/drop markers, live vehicle.
Charts	Recharts (or Chart.js)	Reports dashboard: trips, distance, fuel, cost/km.
Backend	Python + FastAPI	Async, native WebSockets, auto OpenAPI docs — ideal for REST + real-time.
ORM / migrations	SQLAlchemy 2.0 + Alembic (or SQLModel)	Typed models matching this schema; versioned migrations.
Validation	Pydantic v2	Request/response schemas, tight with FastAPI.
Real-time	FastAPI WebSockets (or python-socketio) + Redis pub/sub	Live location pings + chat; Redis fans out across workers.
Database	PostgreSQL 15 (+ PostGIS for matching)	Relational fit; PostGIS makes proximity ride-matching fast.
Auth	JWT access/refresh (python-jose) + passlib[bcrypt]	Stateless; role guard for admin vs employee.
Payments	Razorpay Test Mode (razorpay Python SDK)	Orders API + Checkout + webhook; sandbox, no real money.
Background jobs	FastAPI BackgroundTasks / Celery (optional)	Recurring-ride generation, notification fan-out.
Dev / run	Docker Compose (Postgres + Redis) + uvicorn	One command locally; no cloud hosting required.

Django alternative: Django REST Framework + Channels + Django ORM is equally valid — and its built-in admin panel can double as the Company Administrator console, saving UI time. Trade-off: Channels real-time setup is heavier than FastAPI's native WebSockets.

6 · External APIs & Services per Feature

Researched options per feature with a recommendation. Bias: free / sandbox tiers, works on localhost, India-friendly (the brief mandates Razorpay / INR). You confirm before we lock these in.

Feature	Options	Recommended (free / sandbox)
Map display	OpenStreetMap tiles · Mapbox · Google Maps	OSM tiles via React-Leaflet — no key, free.
Address search / autocomplete	OSM Nominatim · Photon · Google Places · LocationIQ · Mappls (India)	Photon or LocationIQ (free tier, autocomplete). Google Places if budget allows.
Geocoding (address ↔ lat/lng)	Nominatim · LocationIQ · OpenCage · Google Geocoding	Nominatim (self-host/demo) or LocationIQ free tier.
Routing / distance / ETA	OSRM · OpenRouteService · GraphHopper · Mapbox Directions · Google Directions	OSRM (self-host, unlimited, no key) or OpenRouteService free tier.
Route confirmation polyline	Same routing engine returns encoded polyline	OSRM /route → geometry; draw with Leaflet.
Live trip tracking	Browser Geolocation API → your WebSocket · third-party fleet APIs	Browser Geolocation + FastAPI WebSocket — no external cost.
Ride matching	In-DB PostGIS query · custom scoring	PostGIS ST_DWithin on origin/dest + time window; internal.
Payments (card/UPI/wallet-recharge)	Razorpay · Stripe · Cashfree · PayU	Razorpay Test Mode — mandated by the brief.
In-app chat	Your WebSocket · Firebase · Stream · Sendbird	FastAPI WebSocket + messages table — no external cost.
Voice call	Twilio · Agora · Exotel (India) · plain tel: dial · WebRTC	Demo: tel: deep-link to stored phone (free). Real masked calling: Exotel (IN) or Twilio.
Push notifications (bonus)	Firebase Cloud Messaging · WebSocket in-app · OneSignal	In-app via WebSocket; FCM (free) if you want true mobile push.
Email (verify / receipts)	SMTP · SendGrid · Resend · Mailtrap (test)	Mailtrap sandbox for dev; SendGrid/Resend free tier for real.
Image upload (profile / vehicle)	Local disk · Cloudinary · AWS S3 · Supabase Storage	Local disk for demo; Cloudinary free tier if hosted.

The one paid decision: Voice Call. Everything else is free/sandbox and localhost-friendly. For the demo, a tel: dial link costs nothing; only invest in Exotel/Twilio masked calling if hiding phone numbers is a scoring criterion.

Suggested build order for the hackathon

1. Auth + org/user seed data (login, signup, roles, status).
2. Document upload at signup + admin verification console (licence → verified).
3. Vehicle management (incl. mileage) + saved places.
4. Offer a ride → gate on verified licence + active vehicle → route confirmation (Leaflet + OSRM) → publish.
5. Find a ride → PostGIS matching query → booking (seat decrement in a txn).
6. My Trips + trip lifecycle state machine (scheduled→started→in_progress→completed).
7. Live tracking via WebSocket + chat.
8. Payments (Razorpay test) + wallet ledger + recharge.
9. Reports dashboard (trips, distance, fuel, cost/km) + bonus features.